CYBERSECURITY GUIDE BOOK

CHAPTER 1

Networking

OSI | IP addressing | Subnetting | TCP/IP | DNS | DHCP | HTTP/HTTPS | TLS

Roadmap step 1 of 12

Master the fundamentals first, then specialize.
Every concept is shown from two sides: how an attacker abuses it,
and how a defender (SOC analyst) detects and stops it.

Author: ashihh07

For learning and authorized lab use only.

About this chapter

Every attack travels over a network, and almost every detection a SOC analyst works with is built from network data: firewall logs, DNS logs, proxy logs, packet captures. If you cannot read an IP range, a port number or a packet header, you cannot decide whether an alert is real. This chapter gives you that vocabulary, plus the commands to prove each idea on your own machine.

How each topic is built

- **Concept** - what it is and why it exists, in plain language.
- **Attacker view** - how the concept gets abused (for understanding and authorized testing only).
- **Defender view** - what to log, what to look for, and which control stops it.
- **Commands and labs** - everything is testable on a Linux machine or VM you own.

Contents

#	Section	Roadmap item covered
1	The OSI model and TCP/IP model	OSI Model
2	IP addressing (IPv4, private ranges, IPv6 basics)	Subnetting, IP Addressing
3	Subnetting and CIDR	Subnetting, IP Addressing
4	TCP, UDP, ports, ICMP and ARP	TCP/IP
5	DNS	DNS
6	DHCP	DHCP
7	HTTP	HTTP/HTTPS
8	TLS (formerly SSL)	SSL/TLS
9	Hands-on labs	Practice for all of the above
10	Quick revision sheet	Commands, ports, subnet table
11	Self-check quiz and answers	Test yourself

Ground rule for every lab in this book

- Only scan, capture or test systems you own or have written permission to test.
- Use your own VM, home lab or a platform built for practice (TryHackMe, Hack The Box).
- Unauthorized scanning can be illegal even if you mean no harm.

Study order

The sections build on each other. OSI gives you the map, addressing and subnetting give you locations on that map, TCP/UDP describe how conversations work, and DNS, DHCP, HTTP and TLS are the services you will meet in almost every alert. Do them in order, and do the labs before moving on.

1. The OSI model and TCP/IP model

Concept

A network is a stack of jobs. The **OSI model** splits those jobs into seven layers so that people can talk about problems precisely: "it is a layer 2 issue" tells a colleague exactly where to look. Each layer only talks to the layer above and below it, and each adds its own header to the data.

#	Layer	Data unit	Examples	Typical attacks
7	Application	Data	HTTP, DNS, SMTP, SSH	Phishing, SQL injection, XSS
6	Presentation	Data	TLS, encoding, compression	Downgrade attacks, bad certificates
5	Session	Data	Sessions, RPC, NetBIOS	Session hijacking
4	Transport	Segment (TCP) / Datagram (UDP)	TCP, UDP, ports	SYN flood, port scanning
3	Network	Packet	IP, ICMP, routers	IP spoofing, ICMP flood
2	Data Link	Frame	Ethernet, Wi-Fi, ARP, switches	ARP spoofing, MAC flooding
1	Physical	Bits	Cables, radio, hubs	Wiretapping, jamming, cable cut

Memory trick (layer 1 up to 7): *Please Do Not Throw Sausage Pizza Away* = Physical, Data Link, Network, Transport, Session, Presentation, Application.

TCP/IP model: what is actually used

The real internet runs on the simpler **TCP/IP model** (four layers). OSI is the teaching model; TCP/IP is the working one. You need both, because interviews use OSI wording and packet captures show TCP/IP reality.

TCP/IP layer	OSI layers	Protocols
Application	7, 6, 5	HTTP, HTTPS, DNS, DHCP, SSH, SMTP, FTP
Transport	4	TCP, UDP
Internet	3	IP, ICMP
Network Access (Link)	2, 1	Ethernet, Wi-Fi, ARP

Encapsulation: how data is wrapped

When you open a web page, your request is wrapped layer by layer on the way out and unwrapped in reverse on the way in. Wireshark shows exactly these layers, one nested inside the next.

```
Application data   : GET /index.html HTTP/1.1
+ TCP header       : src port 51514, dst port 80, seq/ack numbers, flags   = segment
+ IP header        : src 192.168.1.10, dst 203.0.113.50, TTL                = packet
+ Ethernet header  : src MAC, dst MAC (next hop), FCS trailer               = frame
+ Physical signal  : electrical pulses / light / radio                      = bits
```

Defender view: why layers matter

- Controls live at specific layers: switch port security (L2), firewalls and ACLs (L3/L4), web application firewalls (L7). Knowing the layer tells you which control should have stopped an attack.
- Troubleshooting order: start at layer 1 (cable, link light) and move up. Most "mystery" outages are layer 1-3 problems.

2. IP addressing

Concept

An **IPv4 address** is a 32-bit number written as four decimal octets (0-255), for example `192.168.1.10`. It identifies a host on a network, like a postal address. A **MAC address** is different: it identifies the network card on the local link (layer 2) and does not travel past your router. IP is end-to-end, MAC is hop-to-hop.

Binary conversion (you need this for subnetting)

Each octet is 8 bits with place values 128, 64, 32, 16, 8, 4, 2, 1. Add the values that are switched on.

Octet	128	64	32	16	8	4	2	1	Binary
192	1	1	0	0	0	0	0	0	11000000
168	1	0	1	0	1	0	0	0	10101000
10	0	0	0	0	1	0	1	0	00001010
1	0	0	0	0	0	0	0	1	00000001

Address types you must recognise on sight

Range	Meaning	Why it matters for security
10.0.0.0/8	Private (RFC 1918)	Internal networks. Seeing one as a source on the internet side is suspicious.
172.16.0.0/12	Private (172.16.0.0 to 172.31.255.255)	Common in corporate and cloud networks.
192.168.0.0/16	Private (RFC 1918)	Home and small office networks.
127.0.0.0/8	Loopback (127.0.0.1 = this machine)	Traffic that never leaves the host. Used to test local services.
169.254.0.0/16	Link-local (APIPA)	Host asked for DHCP and got no answer. A sign DHCP is broken.
0.0.0.0	"This host" / default route	A service bound to 0.0.0.0 listens on all interfaces. Check exposure.
224.0.0.0/4	Multicast	Discovery protocols. Noisy but normally harmless.
Everything else	Public, routable on the internet	The addresses you block, geolocate and look up in threat intelligence.

NAT (Network Address Translation) lets many private hosts share one public IP. In logs this means the public address you see at the perimeter is the router, not the real internal host. You need the NAT or firewall table to find which internal machine made the connection.

IPv6 in one page

- 128-bit addresses in hex, eight groups: `2001:db8:85a3::8a2e:370:7334` (`::` replaces one run of zero groups).
- `::1` is loopback. `fe80::/10` is link-local. `fc00::/7` is private (unique local). `2001:db8::/32` is reserved for documentation.
- No broadcast and no NAT by design. Hosts often configure themselves (SLAAC).

Attacker view

- IPv6 is often enabled by default but ignored by monitoring and firewall rules, so it can be an unwatched path.
- Rogue router advertisements can redirect IPv6 traffic on a local network.
- Spoofed source IPs are used in DDoS reflection and to hide scan origin.

Defender view

- Treat IPv6 like IPv4: same firewall rules, same logging. If you do not use it, disable it deliberately.
- Alert on private source addresses arriving from outside (bogon filtering) and on 169.254.x.x clusters.

3. Subnetting and CIDR

Concept

A **subnet mask** splits an address into a network part and a host part. **CIDR notation** writes the mask as the count of network bits: `192.168.10.0/24` means the first 24 bits are the network, the last 8 bits are for hosts. Subnetting is how you cut a big network into smaller, separated ones.

Three formulas that solve almost every problem

- Host bits = 32 - prefix length.
- Total addresses = $2^{\text{host bits}}$. Usable hosts = total - 2 (network address and broadcast address are reserved).
- Block size = 256 - the interesting octet of the mask. Subnets start at multiples of the block size.

Prefix	Subnet mask	Block size	Total addresses	Usable hosts
/8	255.0.0.0	-	16,777,216	16,777,214
/16	255.255.0.0	-	65,536	65,534
/24	255.255.255.0	256	256	254
/25	255.255.255.128	128	128	126
/26	255.255.255.192	64	64	62
/27	255.255.255.224	32	32	30
/28	255.255.255.240	16	16	14
/29	255.255.255.248	8	8	6
/30	255.255.255.252	4	4	2

Worked example 1: find the network, range and broadcast

Question: What subnet does `192.168.10.77/26` belong to?

```
Mask /26  = 255.255.255.192      -> interesting octet is the 4th (192)
Block size = 256 - 192 = 64      -> subnets start at .0  .64  .128  .192
77 falls between 64 and 127      -> network address = 192.168.10.64
Broadcast = next network - 1     -> 192.168.10.127
Usable range                     -> 192.168.10.65  to  192.168.10.126  (62 hosts)
```

Worked example 2: split one network into equal parts

Question: Split `192.168.1.0/24` into 4 equal subnets.

```
4 subnets = 2^2           -> borrow 2 bits -> new prefix /26
Block size 64             -> 192.168.1.0/26    192.168.1.64/26
                             192.168.1.128/26  192.168.1.192/26
Each subnet: 62 usable hosts
```

Worked example 3: size a subnet from a requirement

Question: A department needs 50 hosts. Which prefix? Smallest 2^h - 2 that is at least 50: h = 6 gives 62. Host bits 6 means prefix 32 - 6 = **/26**.

Practice problems

#	Problem
1	Find network, broadcast and usable range for 10.0.5.130/25.
2	How many usable hosts are in a /27?
3	Find network and broadcast for 172.16.45.200/20.
4	Are 192.168.1.20 and 192.168.1.200 in the same /25?
5	Which prefix gives at least 12 usable hosts?

Answers: (1) network 10.0.5.128, broadcast 10.0.5.255, usable 10.0.5.129 to 10.0.5.254. (2) 30. (3) mask 255.255.240.0, block size 16 in the third octet, 45 sits in 32-47: network 172.16.32.0, broadcast 172.16.47.255. (4) No: .20 is in 0-127, .200 is in 128-255. (5) /28 (14 usable).

Why subnetting is a security skill

- **Segmentation:** separate users, servers, guest Wi-Fi and IoT into their own subnets and VLANs so one compromised device cannot freely reach everything.
- **Firewall rules and SIEM queries use CIDR.** Writing 10.20.30.0/24 instead of 254 addresses is the daily job.
- **Scope:** a scan or an incident always starts with "which range is affected?"

Attacker view

- After landing on a host, an attacker reads its IP and mask to learn the size of the network, then sweeps that range to find neighbours (lateral movement). Flat networks make this easy.

4. TCP, UDP, ports, ICMP and ARP

TCP vs UDP

	TCP	UDP
Connection	Connection-oriented (handshake first)	Connectionless (just send)
Reliability	Acknowledgements, retransmission, ordering	None. The application handles loss
Speed	Slower, more overhead	Faster, lightweight
Used for	Web, SSH, email, file transfer	DNS queries, DHCP, NTP, VoIP, streaming, syslog

The TCP three-way handshake

```
Client                                Server
  |--------- SYN (seq=x) ------------->|   "I want to connect"
  |<-- SYN-ACK (seq=y, ack=x+1) -------|   "OK, I am listening"
  |--------- ACK (ack=y+1) ------------>|   "Connection established"
  |========= data flows ==============|
  |--------- FIN / ACK --------------->|   graceful close (RST = abrupt close)
```

Flags to know: **SYN** start, **ACK** acknowledge, **FIN** finish, **RST** reset/refuse, **PSH** push data now, **URG** urgent. A packet with unusual flag combinations is almost always a scan.

Ports

A port is a 16-bit number (0-65535) that tells the operating system which program gets the traffic. **0-1023** well-known, **1024-49151** registered, **49152-65535** ephemeral (your side of a connection).

Port	Service	Security note
20/21	FTP	Credentials in clear text. Prefer SFTP/FTPS.
22	SSH	Remote login. Constant target of brute force from the internet.
23	Telnet	Clear text remote login. Should not exist on a modern network.
25	SMTP	Mail transfer. Spam relay and phishing delivery.
53	DNS (UDP and TCP)	UDP for normal queries, TCP for large answers and zone transfers.
67/68	DHCP (UDP)	Server listens on 67, client on 68.
80	HTTP	Clear text web.
110 / 143	POP3 / IMAP	Mail retrieval. Use the encrypted variants (995 / 993).
123	NTP (UDP)	Time sync. Abused for DDoS amplification when misconfigured.
161	SNMP (UDP)	Default community strings leak device details.
443	HTTPS	Encrypted web. Attackers hide inside it too.
445	SMB	File sharing. Ransomware and lateral movement favourite.
3306	MySQL	Databases should never face the internet.
3389	RDP	Windows remote desktop. Exposed RDP is a top ransomware entry point.

ICMP and ARP

- **ICMP** (layer 3) carries control messages. `ping` sends echo request (type 8) and gets echo reply (type 0). `traceroute` sends packets with a rising TTL so each router reveals itself when the TTL hits zero.
- **ARP** (layer 2) answers "who has IP 192.168.1.1? tell me its MAC" on the local network. It has no authentication: any host can claim any IP.

Attacker view

- **SYN scan (nmap -sS):** send SYN, read the answer. SYN-ACK means open, RST means closed, silence means filtered by a firewall. No full connection is made, so older logging may miss it.
- **SYN flood:** send a storm of SYNs and never finish the handshake, filling the server's half-open connection table.
- **ARP spoofing:** send fake ARP replies so the victim sends traffic to the attacker (man-in-the-middle).

Defender view

- One source touching many ports or many hosts in a short time is the signature of a scan. Many SYNs with few completed handshakes points to a flood.
- Mitigations: SYN cookies and rate limiting, default-deny firewall rules, Dynamic ARP Inspection and port security on switches, closing every port you do not need.
- Baseline what should be listening: `ss -tulpn` on every server.

5. DNS (Domain Name System)

Concept

DNS translates names people remember (example.com) into IP addresses machines use. It is a distributed, hierarchical database: **root servers**, then **TLD servers** (.com, .in, .org), then the **authoritative server** that owns the domain.

What happens when you type a domain

```
1. Browser/OS cache  -> answer already known? done.
2. Stub resolver asks the recursive resolver (your ISP, router, 8.8.8.8, 1.1.1.1)
3. Recursive resolver asks a root server    -> "ask the .com servers"
4. ... asks the .com TLD server             -> "ask the authoritative server for example.com"
5. ... asks the authoritative server        -> "example.com is 203.0.113.50"
6. Answer is cached for the record's TTL, then returned to your browser
```

Record types

Record	Purpose	Example
A	Name to IPv4 address	example.com -> 203.0.113.50
AAAA	Name to IPv6 address	example.com -> 2001:db8::50
CNAME	Alias to another name	www.example.com -> example.com
MX	Mail servers for the domain	10 mail.example.com
NS	Authoritative name servers	ns1.example.com
TXT	Free text: SPF, DKIM, DMARC, verification	v=spf1 include:... -all
PTR	Reverse lookup: IP to name	50.113.0.203.in-addr.arpa -> host name
SOA	Zone start of authority (serial, timers)	Primary NS, admin contact

Commands

```
dig example.com A              # IPv4 address
dig example.com MX +short      # mail servers, short output
dig @8.8.8.8 example.com NS    # ask a specific resolver
dig +trace example.com         # walk root -> TLD -> authoritative yourself
dig -x 8.8.8.8                 # reverse lookup
nslookup example.com           # same idea, works on Windows too
```

Attacker view

- **Recon:** enumerate subdomains and records to map a target. A misconfigured server may allow a full **zone transfer (AXFR)** that hands over every record.
- **Cache poisoning / spoofing:** make a resolver store a false answer so users reach a fake site.
- **DNS tunneling:** hide data or command-and-control traffic inside very long or odd subdomain queries.
- **Look-alike domains** (typosquatting) and **dangling records** (subdomain takeover) for phishing and hijacking.
- **Real-world:** in October 2016 the Mirai botnet flooded the DNS provider Dyn, and many major sites became unreachable for hours. DNS is critical infrastructure.

Defender view

- **Log DNS queries.** It is one of the best data sources in a SOC: every malware sample needs to resolve something.
- Hunt for: very long or high-randomness subdomains, many TXT queries to one domain, newly registered domains, sudden spikes of NXDOMAIN (domain generation algorithms).
- Controls: restrict zone transfers to known secondaries, DNSSEC where possible, protective DNS / sinkholing, force internal clients to use your resolver only.

6. DHCP (Dynamic Host Configuration Protocol)

Concept

DHCP gives a device its network settings automatically when it joins: IP address, subnet mask, default gateway, DNS servers and a lease time. It runs over UDP (server port 67, client port 68). The exchange is remembered as **DORA**:

```
Client                                    DHCP server
  |-- DISCOVER (broadcast) ----------------->|   "Is there a DHCP server?"
  |<-- OFFER  (192.168.1.50 for 24h) --------|   "You can have this"
  |-- REQUEST (I accept 192.168.1.50) ------>|   "I'll take it"
  |<-- ACK    (lease confirmed + options) ---|   "It is yours: gateway, DNS, mask"
```

Because the first message is a broadcast and nothing is authenticated, the client trusts the first answer it hears. That one fact explains the attacks below.

Attacker view

- **Rogue DHCP server:** answer faster than the real one and hand out the attacker's machine as the gateway or DNS server, giving a man-in-the-middle position.
- **DHCP starvation:** request every address in the pool with fake MACs so legitimate devices get nothing, often as a first step before a rogue server.

Defender view

- **DHCP snooping** on switches: only ports marked trusted may send server replies. This stops rogue servers.
- Port security (limit MACs per port) against starvation. Monitor for a second DHCP server on the network.
- DHCP logs map an IP to a MAC and a time. During an investigation they answer "which device had this IP at 14:32?"

```
ip addr                    # your IP and mask          (Windows: ipconfig /all)
ip route                   # default gateway           (Windows: route print)
resolvectl status          # DNS servers in use
sudo dhclient -v eth0      # watch a DHCP exchange     (Windows: ipconfig /release then /renew)
```

7. HTTP

Concept

HTTP is the request-response protocol of the web (TCP port 80). The client sends a request, the server sends a response. It is **stateless**: the server does not remember you between requests, which is why cookies and sessions exist (covered again in Chapter 4).

```
GET /login?user=alice HTTP/1.1          <- request line: method, path, version
Host: shop.example.com                  <- headers
User-Agent: Mozilla/5.0
Cookie: session=abc123

HTTP/1.1 200 OK                         <- status line
Content-Type: text/html
Set-Cookie: session=xyz789; Secure; HttpOnly; SameSite=Lax
Server: nginx/1.18.0                    <- reveals software and version

<html> ... </html>                      <- body
```

Methods and status codes

Methods	Use
GET / HEAD	Fetch a resource / fetch only the headers
POST / PUT	Send data to create or update
DELETE / OPTIONS	Remove / ask what is allowed

Class	Meaning	Codes worth knowing
2xx	Success	200 OK, 201 Created
3xx	Redirect	301 Permanent, 302 Temporary
4xx	Client error	400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found
5xx	Server error	500 Internal Error, 502 Bad Gateway, 503 Service Unavailable

Security-relevant headers

Header	What it does
Strict-Transport-Security (HSTS)	Tells the browser to use HTTPS only for this domain for a set time.
Content-Security-Policy	Limits where scripts and content may load from. Reduces XSS impact.
X-Content-Type-Options: nosniff	Stops the browser guessing content types.
X-Frame-Options / frame-ancestors	Controls whether the page can be framed (clickjacking defence).
Set-Cookie flags: Secure, HttpOnly, SameSite	Secure: HTTPS only. HttpOnly: hidden from JavaScript. SameSite: limits cross-site sending.

```
curl -I https://example.com       # response headers only
curl -v http://neverssl.com       # full request and response, plain HTTP for practice
```

Attacker view

- Anything sent over plain HTTP (logins, cookies) can be read or altered by anyone on the path.
- Banners like `Server: nginx/1.18.0` tell an attacker which known vulnerabilities to try.
- Parameters in the URL and body are the input for SQL injection, XSS and other web attacks (Chapter 5).

Defender view (reading web logs)

- Many **401/403** from one IP: password guessing or access probing.
- Hundreds of **404** in seconds: a scanner looking for hidden paths.
- Odd **User-Agent** strings (sqlmap, nikto, curl in places it should not be) and long encoded parameters.
- Fixes: HTTPS everywhere plus HSTS, hide version banners, set the security headers above.

8. TLS (formerly SSL)

Concept

TLS wraps a protocol such as HTTP so that the connection gets three things: **confidentiality** (encryption), **integrity** (tampering is detected) and **authentication** (you are talking to the real server). HTTP over TLS is HTTPS (port 443). **SSL is obsolete**: SSL 2.0 and 3.0 are broken, and TLS 1.0 and 1.1 were formally deprecated in 2021. Use **TLS 1.2 or 1.3** only.

How the handshake works (simplified)

```
1. ClientHello : supported TLS versions, cipher suites, random value, key share
2. ServerHello : chosen version and cipher, server certificate, key share
3. Client checks the certificate: trusted chain? not expired? name matches? not revoked?
4. Both sides derive the same session key from the key exchange
5. Everything after this is encrypted with fast symmetric encryption
```

The pattern to remember: **asymmetric cryptography** is used briefly to authenticate and agree on a key, then **symmetric cryptography** encrypts the actual data because it is much faster.

Certificates and the chain of trust

A certificate (X.509) binds a public key to a name. It contains the subject, the Subject Alternative Names (the hostnames it is valid for), the issuer, validity dates and the public key. Your browser trusts a site because the certificate chains up to a root CA already in its trust store:

```
Root CA  (in your OS/browser trust store)
   |  signs
Intermediate CA
   |  signs
Server certificate  (shop.example.com)
```

```
openssl s_client -connect example.com:443 -servername example.com
echo | openssl s_client -connect example.com:443 -servername example.com 2>/dev/null \
  | openssl x509 -noout -subject -issuer -dates
```

Attacker view

- **Man-in-the-middle with a fake or self-signed certificate:** works only if the user clicks through the browser warning or trusts a malicious root certificate.
- **Downgrade and old-protocol attacks:** force or exploit weak versions (for example POODLE against SSL 3.0, 2014).
- **Implementation bugs:** Heartbleed (2014) let attackers read server memory from a vulnerable OpenSSL.
- **SSL stripping:** keep the victim on HTTP while talking HTTPS to the server. HSTS defeats this.

Defender view

- Allow TLS 1.2+ only, remove weak ciphers, enable HSTS, and automate renewal so certificates never expire (expired certificates cause outages and train users to ignore warnings).
- Test any public site with **Qualys SSL Labs** (ssllabs.com/ssltest) and fix what it flags.
- Monitor certificate issuance for your domains and watch for self-signed or unexpected certificates in traffic logs.
- Encrypted traffic is still observable: SNI, certificate fields, timing and volume help a SOC even without decryption.

9. Hands-on labs

Do these on a Linux machine or VM you own. After each lab, save a short write-up in your repo (goal, commands, a screenshot, what you learned). Short and honest beats long and copied.

Lab 1: Subnet by hand, then verify

```
sudo apt install ipcalc
ipcalc 192.168.10.77/26        # compare with your hand answer
ipcalc 172.16.45.200/20
```

Goal: solve the five practice problems on paper first, then check each with ipcalc.

Lab 2: Map your own network

```
ip addr          # my IP, mask, interface
ip route         # default gateway
ip neigh         # ARP table: who is on my LAN
resolvectl status
ss -tulpn        # what is listening on this machine, and which process owns it
```

Goal: write down your subnet, gateway, DNS server and every listening port. Decide whether each listener is expected.

Lab 3: Watch a handshake and a plain-text request

```
sudo tcpdump -i any -n -c 25 'tcp port 80'      # terminal 1
curl http://neverssl.com                        # terminal 2
```

Goal: find SYN, SYN-ACK, ACK and the GET request. Repeat in Wireshark with the display filter `tcp.flags.syn == 1`, then right-click a packet and choose Follow TCP Stream.

Lab 4: Trace a DNS lookup

```
dig +trace example.com
sudo tcpdump -i any -n port 53 -c 10 &          # then run:  dig example.com
```

Goal: identify the root, TLD and authoritative steps, and the TTL on the final answer.

Lab 5: Inspect a TLS certificate

Run the openssl commands from section 8 against two sites. Record issuer, validity dates and hostnames (SANs). Then run both through SSL Labs and note the grade and the first weakness it lists.

Lab 6: Port scan a machine you own

```
sudo nmap -sS -p 22,80,443 192.168.56.101       # your own VM only
sudo nmap -sV -p- 192.168.56.101                # all ports plus service versions
```

Goal: compare nmap's open ports with `ss -tulpn` run on the target. They should match. Capture the scan in Wireshark and look at the SYN, SYN-ACK and RST pattern from section 4.

> **Next step on TryHackMe**
>
> - Work through the networking rooms in the Pre Security path and a Wireshark basics room, then write them up the same way.

10. Quick revision sheet

Commands

Task	Linux	Windows
My IP and interfaces	ip addr	ipconfig /all
Routing table, gateway	ip route	route print
ARP table	ip neigh	arp -a
Listening ports and process	ss -tulpn	netstat -ano
Reachability	ping -c 4 host	ping host
Path to host	traceroute host	tracert host
DNS lookup	dig host / nslookup host	nslookup host
HTTP headers	curl -I url	curl.exe -I url
TLS details	openssl s_client -connect host:443	same (if OpenSSL installed)
Packet capture	sudo tcpdump -i any -n port 53	Wireshark
Port scan (authorized only)	nmap -sS -p- host	nmap -sS -p- host

One-line summaries

- **OSI** 7 layers: Physical, Data Link, Network, Transport, Session, Presentation, Application.
- **Usable hosts** = $2^{\text{host bits}}$ - 2. **Block size** = 256 - mask octet.
- **TCP** SYN, SYN-ACK, ACK. **UDP** no handshake. **DORA** Discover, Offer, Request, Acknowledge.
- **DNS** A = IPv4, AAAA = IPv6, MX = mail, NS = name server, TXT = text/SPF, PTR = reverse.
- **HTTP** stateless, clear text, port 80. **HTTPS** = HTTP + TLS, port 443, TLS 1.2/1.3 only.
- **Attack to control:** ARP spoofing - DAI. Rogue DHCP - DHCP snooping. SYN flood - SYN cookies. SSL strip - HSTS. Zone transfer - restrict AXFR.

11. Self-check quiz

Answer without looking back, then check the key below. Anything you miss goes on your revision list.

1. At which OSI layer does a router mainly operate, and what is the data unit called there?

2. How many usable hosts are in a /29?

3. A laptop shows the address 169.254.12.7. What most likely happened?

4. Put the TCP handshake packets in order.

5. Which port and protocol does a normal DNS query use?

6. What does DORA stand for?

7. Which DNS record maps a name to an IPv6 address?

8. What does HSTS do?

9. Which switch feature mitigates a rogue DHCP server?

10. Why must TLS 1.0 and 1.1 not be used today?

Answer key

- 1. Layer 3 (Network), packets. 2. 6. 3. DHCP failed, so the OS gave itself a link-local address (APIPA).
- 4. SYN, SYN-ACK, ACK. 5. UDP port 53 (TCP 53 for large answers and zone transfers).
- 6. Discover, Offer, Request, Acknowledge. 7. AAAA.
- 8. Tells browsers to use only HTTPS for the domain for a set period, defeating SSL stripping.
- 9. DHCP snooping (trusted ports). 10. Deprecated with known weaknesses; TLS 1.2 and 1.3 are the standard.

What comes next

Chapter 2: Linux Fundamentals. You have just used ip, ss, dig, curl and tcpdump. Next you learn the operating system underneath them: the file system, permissions, bash, SSH, processes and package management.